KPIT | APEX LAB

Signals in Linux

Asynchronous notifications delivered by the kernel to processes

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

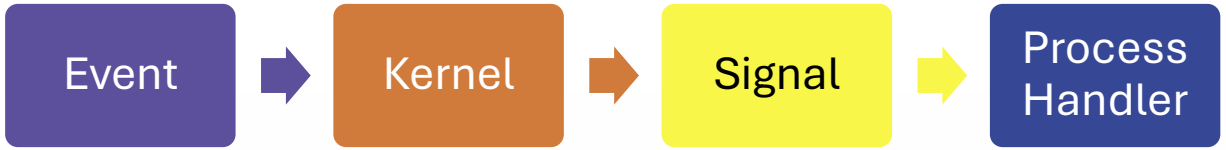
All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Signal

Signals are software interrupts delivered to a process

- Signals interrupt normal program flow.
- They notify a process that some event occurred (e.g., segmentation fault, child exit, timer expiry, CTRL+C, etc.).
- Think of signals as asynchronous notifications sent to a process. Async → can arrive anytime
- #include <signal.h>
- Each signal in Linux is identified by unique number assigned to them
- Used for:
  - Process control
  - Exception handling
  - Inter-process communication
  - Reporting asynchronous events



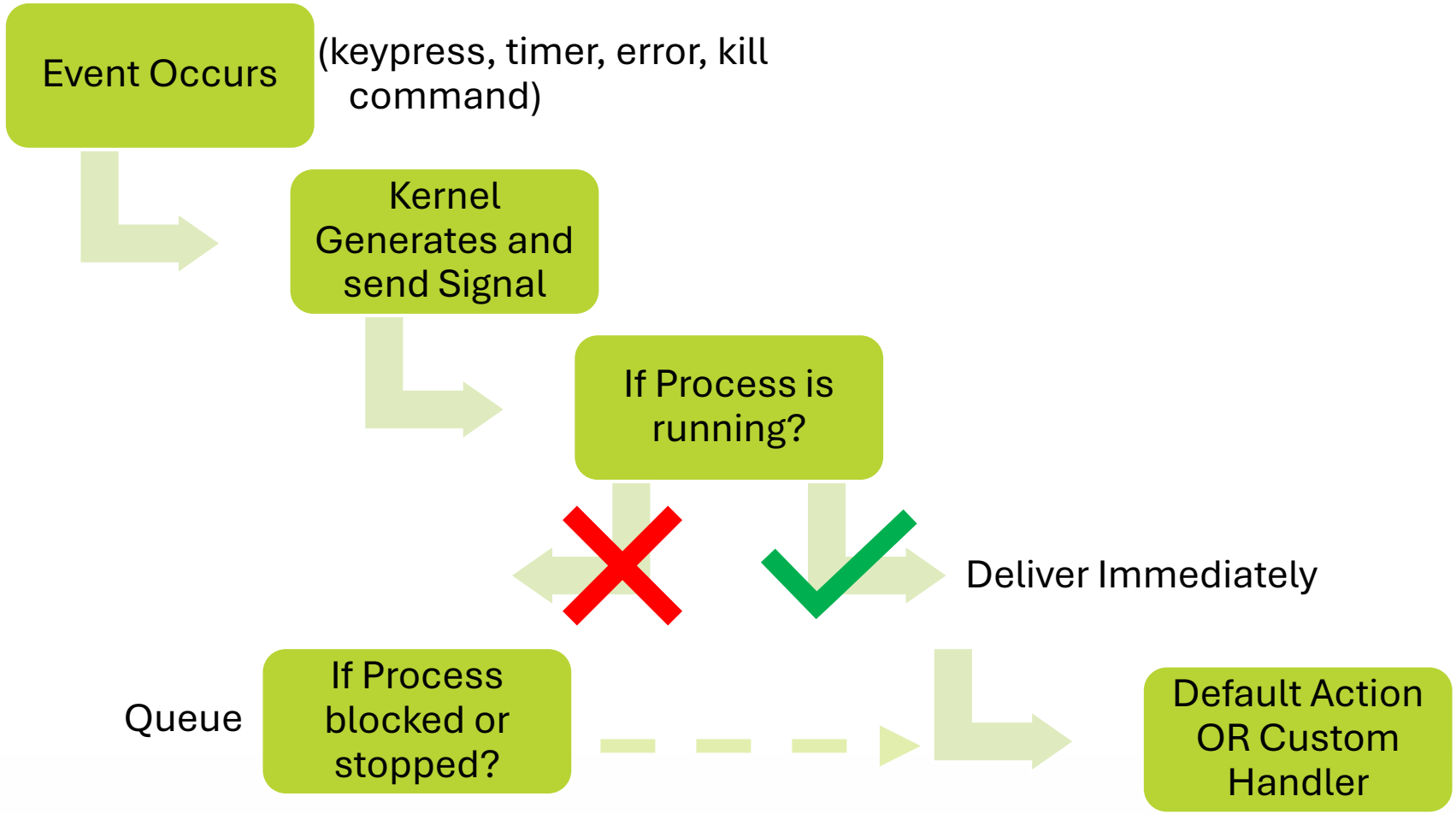
Signal Handling

- Kernel can perform one of the 3 option depending upon what prcess has asked
  - Ignore the signal
  - Catch and handle the signal – Kernel will suspend execution of the process's current code and jumps to a signal handler and the returns to main
  - Perform default action – if there is no user defined signal handler, then default action assigned to that sigal is performed

Common Linux Signals

Signal	Number	Default Action	Meaning
SIGINT	2	Terminate	Ctrl+C
SIGKILL	9	Terminate (cannot be caught/blocked)	Force kill
SIGTERM	15	Terminate	Graceful termination
SIGQUIT	3	Core dump	Ctrl+\
SIGSEGV	11	Core dump	Invalid memory access
SIGABRT	6	Core dump	Abort() called
SIGCHLD	17	Ignore	Child stopped or terminated
SIGSTOP	19	Stop (cannot be caught)	Pause process
SIGCONT	18	Continue	Resume process
SIGALRM	14	Terminate	Alarm timers
SIGPIPE			

Signal Life Cycle



System Calls - Sending Signals

Syntax	Parameters	Return & Errors
signal to process group or PID *int kill(pid_t pid, int sig)*	pid: target process sig: signal number	0 on success -1 on error (ESRCH, EPERM)
Sends signal to itself *int raise(int sig)*	sig: signal number	0 or -1
signal to specific thread id *int tgkill(tgid, tid, sig)*	For threads	0 or -1

System Calls - Installing Signal Handlers

Syntax	Parameters	Return & Errors
sighandler_t signal(int sig, sighandler_t func)	sig: signal func: handler function	Old handler or SIG_ERR
int sigaction(int sig, const struct sigaction *act, struct sigaction *old)	act: new action old: store previous	0 or -1

Catching ctrl+c Signal

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handler(int sig) {
	printf("Caught signal %d\n", sig);
}

int main() {
	signal(SIGINT, handler); // Catch Ctrl+C

	while (1) {
		printf("Running...\n");
		sleep(1);
	}
	return 0;
}
```

- SIGINT comes from terminal driver when user presses Ctrl+C.

- Better: use write() inside handler
- Never call non-reentrant functions from handlers in production code.

Catching Alarm Signal

- alarm() schedules SIGALRM after given seconds.
- pause() is blocked until next signal → good for demo.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void display_message(int s) {
    printf("Generated SIGALARM\n" );
    alarm(2); //for every second
}

int main(void) {
    signal(SIGALRM, display_message);
    alarm(2);

    while (1)
        pause(); // sleep until a signal arrives;
}
```

User-Defined Signals

Signal	Number	Default Action	Usage
SIGUSR1	Typically 10	Terminate	Application-defined event
SIGUSR2	Typically 12	Terminate	Application-defined event

SIGUSR1 / SIGUSR2 are general-purpose signals.

User Signal: Parent Receiving Signal from Child

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>

void handle_signal(int sig) {
    printf("Parent received signal: %d from child\n",
      sig);
}

int main() {
    signal(SIGUSR1, handle_signal);
    // Register signal handler
    pid_t pid = fork();
```

```c
    if (pid < 0) {perror("Fork failed"); exit(1);}

    if (pid == 0) { // Child process
        sleep(2); // Simulate some work
        kill(getppid(), SIGUSR1); // Send signal to
        parent
        exit(0);
    } else { // Parent process
        printf("Parent waiting for signal from
        child...\n");
        pause(); // Wait for signal
        wait();
        printf("Parent exiting.\n");
    }
    return 0;
```

Child → **SIGUSR1** → Parent

Scenario: Log Monitoring System Using SIGUSR1 Between Unrelated Processes

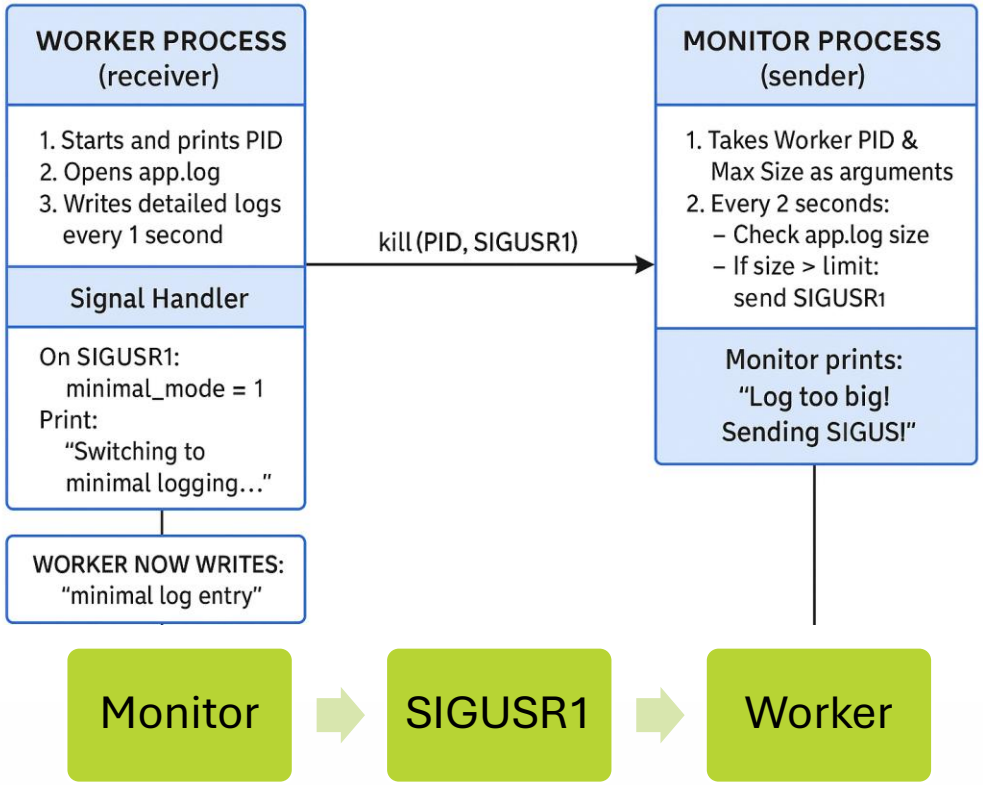
Context:

You are building a simple software setup with two independent processes:

Worker Process: Continuously generates log entries in a file app.log.

Monitor Process: Periodically checks the size of app.log. When the file grows beyond a limit (given by user), the Monitor sends SIGUSR1 to the Worker.

When the Worker receives SIGUSR1, it must switch from detailed logging to minimal logging.



PART 1 — Implement the Worker Process

Create a C file named worker.c

- Print own PID on startup
- Open a file app.log in append mode
- Install a signal handler for SIGUSR1
- Maintain a global flag minimal_mode = 0
- Inside the handler, set minimal_mode = 1
- Keep writing logs every 1 second:
  - If minimal_mode == 0: write “detailed log entry…”
  - Else: write “minimal log entry”

PART 2 — Implement the Monitor Process

Create a C file named monitor.c

- Take two command-line arguments:
  - PID of worker
  - Maximum allowed file size in bytes
- Every 2 seconds:
  - Check size of app.log using stat()
  - If file size > limit → print a message and send SIGUSR1 to worker using kill(pid, SIGUSR1)

Async-Signal-Safe Functions - What you can safely call in a handler

Handlers run asynchronously—call only async-signal-safe functions.c

- A signal handler can interrupt code anytime → avoid non-reentrant functions.
- Only call async-signal-safe functions (e.g., write, _exit, sigatomic-based flag set).
- Avoid: printf, malloc/free, pthread_mutex_*, std::iostream, most library calls.
- Use a global volatile sig_atomic_t flag to communicate from handler → main loop checks it.
- For printing/logging: buffer in main; handler only sets a flag and returns fast.

Common safe examples:

_exit, write, signal(with caveats), sigaction, sigaddset/sigemptyset, kill, alarm, pause, sleep

Blocking & Masking Signals

Temporarily block signals to protect critical sections.

- Blocking ≠ ignoring: delivery is deferred while blocked; pending until unblocked.
- Typical pattern:

```
sigemptyset(&set); sigaddset(&set, SIGINT);
↓
sigprocmask(SIG_BLOCK, &set, &old);   // enter critical section
↓
critical section
↓
sigprocmask(SIG_SETMASK, &old, NULL); // exit critical
```

- Handlers + masks: install with sigaction; you can block specific signals during a handler via sa_mask.

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    sigset_t set, old; sigemptyset(&set); sigaddset(&set, SIGINT);

    // Block SIGINT for 5 seconds (simulate critical section)
    sigprocmask(SIG_BLOCK, &set, &old);

    puts("Critical section: try Ctrl+C now (will be pending)...");
    sleep(5);

    sigset_t pend; sigpending(&pend);
    if (sigismember(&pend, SIGINT)) puts("SIGINT is pending");

    // Unblock (delivery happens here if pending)
    sigprocmask(SIG_SETMASK, &old, NULL);
    puts("Unblocked. If you pressed Ctrl+C, SIGINT should deliver now.");
    pause(); // wait for any signal to observe delivery
}
```

Real-Time Signals (SIGRTMIN … SIGRTMAX)

- **Range**: SIGRTMIN … SIGRTMAX (implementation-defined count; typically 32).
- **Queued**: multiple occurrences are queued (not coalesced).
- **Priority**: higher numbers typically have higher delivery priority.
- **Payload**: use sigqueue() to send with an integer/ptr (union sigval).
- **Install** with sigaction + SA_SIGINFO to read siginfo_t (sender pid, value, etc.).
- **Great for**: event counts, lightweight notifications between processes/threads.
- **Caution**: Still subject to async-signal rules; keep handlers minimal.

```
    pid_t pid = atoi(argv[1]);
    union sigval v; v.sival_int = 42;            // payload
    sigqueue(pid, SIGRTMIN, v);
    return 0;
}


// Receiver
#include <signal.h>
#include <stdio.h>
#include <unistd.h>


static void rt_handler(int sig, siginfo_t *si, void *ctx) {
    (void)ctx;
    // Only async-safe printing is write(); here we just show concept for class:
    printf("Got RT signal %d with value=%d from pid=%d\n",
           sig, si->si_value.sival_int, si->si_pid);
}


int main(void) {
    struct sigaction sa = {0};
    sa.sa_sigaction = rt_handler;
    sa.sa_flags = SA_SIGINFO;
```

signal() vs sigaction()

- signal() — legacy, inconsistent semantics across systems; limited control.
- sigaction() — modern, reliable, supports SA_SIGINFO, masks, flags (SA_RESTART, etc.).
- Rule of thumb: use sigaction() in all new code.